2025

Data Engineering Project- Airline on-time performance analysis

CIS 9760- BIG DATA TECHNOLOGIES

EVELYN NYASEMBI

| evelyn.nyasembi@baruchmail.cuny.edu

Milestone 1

Proposal for Airline Data Analysis

Dataset Description
The dataset contains records of airline performance, including details on flight schedules, delays, cancellations, and other operational metrics. This dataset can be leveraged to analyze trends in airline efficiency, punctuality, and factors influencing delays and cancellations.

Data Source
The dataset is available for download at the following location: Airline on-time performance analysis

Dataset Attributes
The dataset includes 29 columns that capture relevant flight details. Attributes are:

1. **Year** – 1987-2008
2. **Month** – 1-12
3. **DayofMonth** – 1-31
4. **DayofWeek** – 1(Monday) – 7(Sunday)
5. **DepTime** – actual depature time (local, hhmm)
6. **CRSDepTime** – scheduled departure time (local, hhmm)
7. **ArrTime** – actual arrival time (local, hhmm)
8. **CRSArrTime** – scheduled arrival time (local,hhmm)
9. **UniqueCarrier** – unique carrier code
10. **Flight Number** – flight number
11. **Tailnum** – plane tail number
12. **ActualElapsedTime** – in minutes
13. **CRSElapsedTime** – in minutes
14. **AirTime** – in minutes
15. **ArrDelay** – arrival delay, in minutes
16. **DepDelay** – departure delay, in minutes
17. **Origin** – origin IATA airport code
18. **Dest** – destination IATA airport code
19. **Distance** – in miles
20. **TaxiIn** – tax in time, in minutes
21. **TaxiOut** – taxi out time in minutes
22. **Cancelled** – was the flight cancelled
23. **CancellationCode** – reason for cancellation (A = carrier, B = weather, C = NAS, D = security)
24. **Diverted** – 1 = yes, 0 = no
25. **CarrierDelay** – in minutes
26. **WeatherDelay** – in minutes
27. **NasDelay** – in minutes
28. **SecurityDelay** – in minutes
29. **LateAircraftDelay** – in minutes

Prediction Objective:

The goal of this analysis is to predict flight delays using machine learning techniques. Based on the nature of the data and the objectives, **logistic regression** will be used to classify flights as "On Time" or "Delayed" based on whether the delay exceeds a certain percentage (X%) of the total trip time. This method is suitable for categorizing flights effectively while allowing for different delay thresholds (e.g., 10%, 25%, 50%).

Machine Learning Approach

By implement logistic regression to predict whether a delay exceeds a given percentage of total trip time. Additionally, we will experiment with different threshold values (10%, 25%, and 50%) to assess how they impact classification accuracy. Other models such as decision trees or random forests may also be explored for improved performance.

In conclusion, this analysis will provide valuable insights into flight delays, helping airlines and passengers make more informed travel decisions. By identifying key contributing factors to develop better strategies to minimize disruptions in air travel.

Milestone 2

Summary

The data acquisition and storage process involved collecting, storing, and organizing data for further processing. The first step was fetching the dataset from **Kaggle** using a shell script with the curl command. The dataset archive (airline-data.zip) was downloaded directly from the Kaggle repository. After downloading, the dataset was extracted using the unzip command. The extracted files included airline.csv.shuffle and carriers.csv, which were required for analysis.

A **Google Cloud Storage (GCS) bucket** named my-bigdata-project-en was created to store the dataset. The bucket was configured with **standard storage class** and **uniform bucket-level access** to manage permissions and access. Authentication was completed using the gcloud auth login command, allowing interaction with the storage bucket. The extracted dataset files were uploaded to the /landing folder in the GCS bucket using the gcloud storage cp command. A verification step was performed to confirm that the files were successfully transferred to the GCS bucket. The contents of the bucket were listed using gcloud commands.

Additional empty folders were created in the GCS bucket for data processing. These folders included **cleaned**, **trusted**, **code**, and **models**. Empty placeholder files were uploaded to these folders using the gcloud storage cp /dev/null command. These steps established a structured process for data acquisition and storage, ensuring the availability of raw data for further analysis. The structured storage allowed for efficient data processing and integration into subsequent workflows.

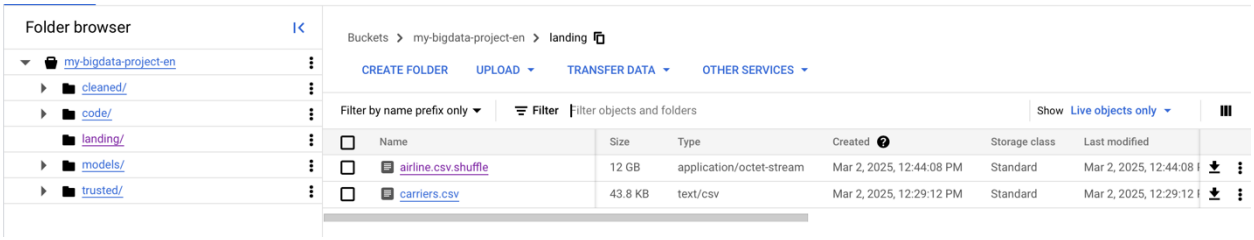
Appendix A Data Acquisition Script

Bucket name:my-bigdata-project-en

1. Fetch the archive file from Kaggle

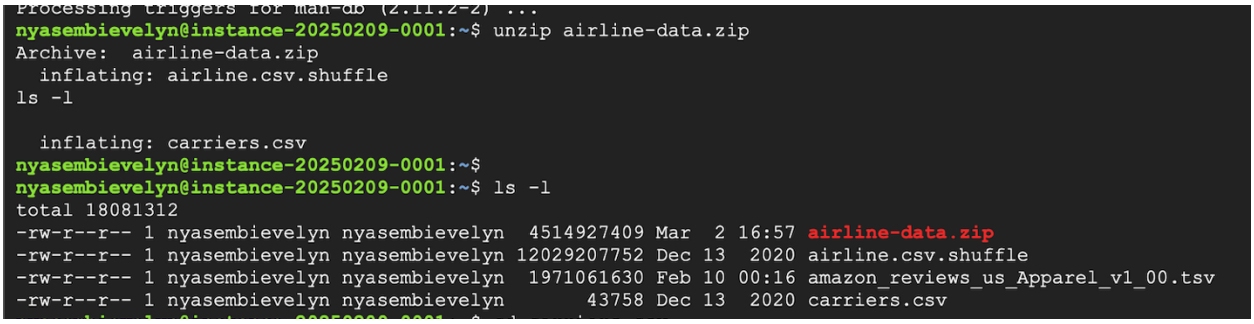
```
#!/bin/bash

curl -L -o airline-data.zip
https://www.kaggle.com/api/v1/datasets/download/bulter22/airline-data
```



2. Unzip the archive file

```
unzip airline-data.zip
```



3. Create a bucket on Google Cloud Storage

```
gcloud storage buckets create gs://my-bigdata-project-en --
project= evelyn-nyasembi-project-1 --default-storage-
class=STANDARD --location=us-central1 --uniform-bucket-level-
access
```

4. Using gcloud auth login to authenticate

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage buckets create gs://my-bigdata-project-en  --project=evelyn-nyasembi-project-1 --default-storage-class=STANDARD --location=us-central1 --uniform-bucket-level-access
Creating gs://my-bigdata-project-en/...
ERROR: (gcloud.storage.buckets.create) HTTPError 403: Access denied. This command is authenticated as 368204563733-compute@developer.gserviceaccount.com which is the active account specified by the [core/account] property.
nyasembievelyn@instance-20250209-0001:~$ gcloud auth login

You are running on a Google Compute Engine virtual machine.
It is recommended that you use service accounts for authentication.

You can run:

  $ gcloud config set account `ACCOUNT`

to switch accounts if necessary.

Your credentials may be visible to others with access to this
virtual machine. Are you sure you want to authenticate with
your personal account?

Do you want to continue (Y/n)?  Y

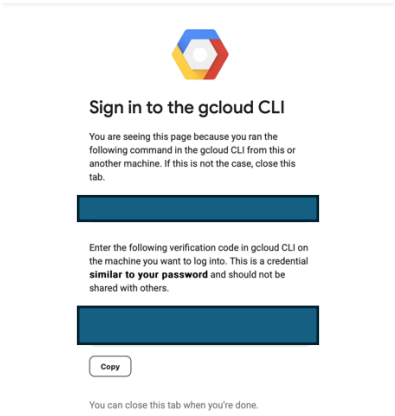
Go to the following link in your browser, and complete the sign-in prompts:

    https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=https%3A%2F%2Fsdk.cloud.google.com%2Fauthcode.html&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-platform+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Faccounts.reauth&state=iZNVOUHL6Fvs7jsDV6vqQybUuEWpyc&prompt=consent&token_usage=remote&access_type=offline&code_challenge=pLuRRW03FLflQFR8ewb2hPZ5X3HyBUcmBptP2j4Xa80&code_challenge_method=S256

Once finished, enter the verification code provided in your browser: 4/0AQSTgQHyf6Rl1M2lxCKH3oZ_fiA9N2CDR69ZMZuxQ_wzyLIR4u41Y85IDUvQgMpfp14H3w

You are now logged in as [nyasembievelyn@gmail.com].
Your current project is [evelyn-nyasembi-project-1].  You can change this setting by running:
  $ gcloud config set project PROJECT_ID


To take a quick anonymous survey, run:
  $ gcloud survey
```



5. Copy the files to GCS bucket in the /landing folder

gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp airline.csv.shuffle gs://my-bigdata-project-en/landing/
WARNING: Parallel composite upload was turned ON to get the best performance on
uploading large objects. If you would like to opt-out and instead
perform a normal upload, run:
`gcloud config set storage/parallel_composite_upload_enabled False`
If you would like to disable this warning, run:
`gcloud config set storage/parallel_composite_upload_enabled True`
Note that with parallel composite uploads, your object might be
uploaded as a composite object
(https://cloud.google.com/storage/docs/composite-objects), which means
that any user who downloads your object will need to use crc32c
checksums to verify data integrity. gcloud storage is capable of
computing crc32c checksums, but this might pose a problem for other
clients.

Copying file://airline.csv.shuffle to gs://my-bigdata-project-en/landing/airline.csv.shuffle
  Completed files 32/1 | 11.2GiB/11.2GiB | 155.5MiB/s

Average throughput: 149.1MiB/s
```

gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage cp carriers.csv gs://my-bigdata-project-en/landing/
Copying file://carriers.csv to gs://my-bigdata-project-en/landing/carriers.csv
  Completed files 1/1 | 42.7kiB/42.7kiB
```

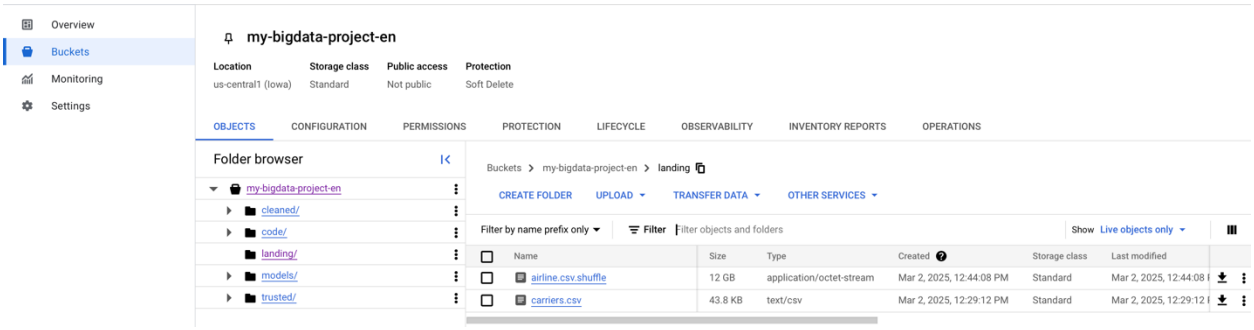
6. List files in gcloud storage

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en/landing
12029207752  2025-03-02T17:44:08Z  gs://my-bigdata-project-en/landing/airline.csv.shuffle
     43758  2025-03-02T17:29:12Z  gs://my-bigdata-project-en/landing/carriers.csv
TOTAL: 2 objects, 12029251510 bytes (11.20GiB)
```

7. Create empty folders

```
gcloud storage cp /dev/null gs://my-bigdata-project-en/cleaned
gcloud storage cp /dev/null gs:// my-bigdata-project-en/trusted
gcloud storage cp /dev/null gs:// my-bigdata-project-en/code
gcloud storage cp /dev/null gs:// my-bigdata-project-en/models
```

```
nyasembievelyn@instance-20250209-0001:~$ gcloud storage ls -l gs://my-bigdata-project-en
                                 gs://my-bigdata-project-en/cleaned/
                                 gs://my-bigdata-project-en/code/
                                 gs://my-bigdata-project-en/landing/
                                 gs://my-bigdata-project-en/models/
                                 gs://my-bigdata-project-en/trusted/
```



Milestone 3

Summary and Issues predicted for feature engineering

In my data analysis project, the first step was to focus on **data cleaning** to ensure the dataset was in a usable state for analysis and machine learning modeling. The dataset contained critical flight information, including **arrival and departure delays**, **flight distances**, and **airports**. I started by removing **invalid values**, specifically filtering out rows where ActualElapsedTime, DepDelay, and ArrDelay were zero or negative, as they don't make sense in the context of flight delays.

Next, I handled **missing values** by dropping rows with null values in key columns like ArrDelay, DepDelay, Cancelled, Distance, Month, Year, and DayOfWeek. This was crucial to ensure that the analysis would be based on complete and reliable data. I also decided to **drop irrelevant columns** such as AirTime, ArrTime, DepTime, and others that weren't contributing to the prediction of delays, which helped simplify the dataset.

Additionally, I made sure that the columns were correctly formatted by **casting them to appropriate data types**. For example, I cast the ArrDelay and DepDelay columns to float to ensure they could be used for calculations like summing delays or calculating averages. This step was necessary before performing any aggregation or analysis.

However, even after cleaning the data, the next step of **feature engineering** presents a few challenges. One major challenge is handling **time-based features** like Month, DayOfWeek, and Year, which can have a significant impact on delay predictions. Deciding how to encode these features to capture trends across different seasons or times of the year will be important for the model.

Another challenge is dealing with **categorical features** such as Origin, Dest, and FlightNum. These will need to be encoded appropriately (e.g., using one-hot encoding or label encoding) to be useful for machine learning models. Finally, the dataset may suffer from **data imbalance**, where there are many more on-time flights than delayed ones. This can lead to biased predictions, so I'll need to consider techniques to address this imbalance, such as resampling or adjusting class weights.

In summary, while I've successfully cleaned the dataset, the challenges in feature engineering, such as handling time-based and categorical features and addressing data imbalance will need to be carefully managed to ensure the model's accuracy in predicting flight delays.

Appendix B Source code

1. Using a 5 cluster dataproc node

gcloud dataproc clusters create cluster-a8fe --enable-component-gateway --region us-central1 --master-machine-type n2-standard-4 --master-boot-disk-type pd-balanced --master-boot-disk-size 100 --num-workers 2 --worker-machine-type n2-standard-4 --worker-boot-disk-type pd-balanced --worker-boot-disk-size 100 --image-version 2.2-debian12 --optional-components JUPYTER --max-idle 3600s –project ProjectID

2. Using PySpark, MatplotLib and Seaborn
    a. Getting data

[6]:
```
# Set the logging level to only show errors
sc.setLogLevel("ERROR")
```

[7]:
```
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"
```

[9]:
```
# Create a Spark Dataframe from some Amazon reviews data
sdf = spark.read.csv(landing_folder, sep=',', header=True, inferSchema=True)
```

b. Schema

```
# Check the schema
sdf.printSchema()
```

```
root
 |-- ActualElapsedTime: string (nullable = true)
 |-- AirTime: string (nullable = true)
 |-- ArrDelay: string (nullable = true)
 |-- ArrTime: string (nullable = true)
 |-- CRSArrTime: integer (nullable = true)
 |-- CRSDepTime: integer (nullable = true)
 |-- CRSElapsedTime: string (nullable = true)
 |-- CancellationCode: string (nullable = true)
 |-- Cancelled: integer (nullable = true)
 |-- CarrierDelay: string (nullable = true)
 |-- DayOfWeek: integer (nullable = true)
 |-- DayofMonth: integer (nullable = true)
 |-- DepDelay: string (nullable = true)
 |-- DepTime: string (nullable = true)
 |-- Dest: string (nullable = true)
 |-- Distance: string (nullable = true)
 |-- Diverted: integer (nullable = true)
 |-- FlightNum: integer (nullable = true)
 |-- LateAircraftDelay: string (nullable = true)
 |-- Month: integer (nullable = true)
 |-- NASDelay: string (nullable = true)
 |-- Origin: string (nullable = true)
 |-- SecurityDelay: string (nullable = true)
 |-- TailNum: string (nullable = true)
 |-- TaxiIn: string (nullable = true)
 |-- TaxiOut: string (nullable = true)
 |-- UniqueCarrier: string (nullable = true)
 |-- WeatherDelay: string (nullable = true)
 |-- Year: integer (nullable = true)
```

c. Number of records

```
# Display the number of records
record_count = sdf.count()
print(f'Total number of records: {record_count}')
```

```
[Stage 2:=============================================================>(89 + 1) / 90]
Total number of records: 123536460
```

d. List of variables

```
# Display the list of columns
columns = sdf.columns
print(f'List of Variables: {columns}')
```

```
List of Variables: ['ActualElapsedTime', 'AirTime', 'ArrDelay', 'ArrTime', 'CRSArrTime', 'CRSDepTime', 'CRSElapsedTime', 'CancellationCode', 'Cancelled', 'Car
rierDelay', 'DayOfWeek', 'DayofMonth', 'DepDelay', 'DepTime', 'Dest', 'Distance', 'Diverted', 'FlightNum', 'LateAircraftDelay', 'Month', 'NASDelay', 'Origin',
'SecurityDelay', 'TailNum', 'TaxiIn', 'TaxiOut', 'UniqueCarrier', 'WeatherDelay', 'Year']
```

e. Filter the data and check count

[9]:
```
# Filter out rows where departure or arrival delay is 0 or negative
sdf = sdf.filter(sdf['ActualElapsedTime'] > 0)
sdf = sdf.filter(sdf['DepDelay'] > 0)
sdf = sdf.filter(sdf['ArrDelay'] > 0)
```

[19]:
```
# Get the new number of records after dropping
print("After filtering and droping missing values")
sdf.count()
```

```
After filtering and droping missing values
```

[19]:
```
37588884
```

f. Count of Missing values

```
]: # Count missing values per column
na_counts = sdf.select(
    [(F.count(F.when(F.col(c).isNull() | (F.col(c) == 'NA'), c)).alias(c)) for c in sdf.columns]
)

# Display the result
na_counts.show()
```

```
[Stage 23:=========================================================>(89 + 1) / 90]
+----------------+--------+--------+--------+----------+----------+--------------+----------------+---------+-------------+---------+----------+--------+--------+
|ActualElapsedTime| AirTime|ArrDelay|ArrTime|CRSArrTime|CRSDepTime|CRSElapsedTime|CancellationCode|Cancelled|CarrierDelay|DayOfWeek|DayofMonth|DepDelay|DepTime|Dest|Distance|Diverted|FlightNum|LateAircraftDelay|Month|NASDelay|Origin|SecurityDelay| TailNum|  TaxiIn| TaxiOut|UniqueCarrier|WeatherDelay|Year|
+----------------+--------+--------+--------+----------+----------+--------------+----------------+---------+-------------+---------+----------+--------+--------+----+--------+--------+---------+-----------------+-----+--------+------+-------------+--------+--------+--------+-------------+------------+----+
|         2587530|39266398| 2589020|2585969|      1491|      1491|         27725|       122801754|     1491|    89330924|     1491|      1491| 2303627|230362|7|1491|  203491|    1491|     1491|         89330924| 1491|89330924|  1491|     89330924|37386911|37398786|37384195|         1491|    89330924|1491|
+----------------+--------+--------+--------+----------+----------+--------------+----------------+---------+-------------+---------+----------+--------+--------+----+--------+--------+---------+-----------------+-----+--------+------+-------------+--------+--------+--------+-------------+------------+----+
```

g. Min/Max Dates

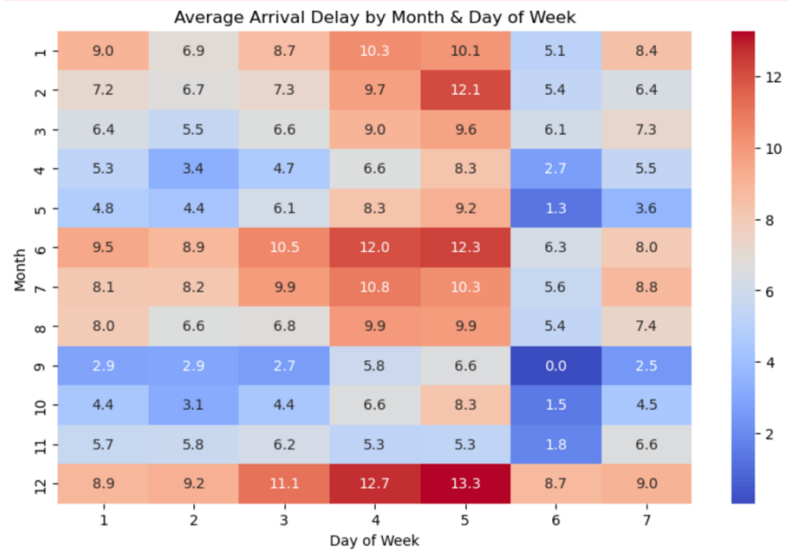
```
[11]: # min and max dates.
sdf.select("DayOfWeek", "DayofMonth", "Month","Year").summary("count", "min", "max").show()
```

```
[Stage 5:=========================================================>(89 + 1) / 90]
+-------+----------+-----------+--------+--------+
|summary|DayOfWeek|DayofMonth|   Month|    Year|
+-------+----------+-----------+--------+--------+
|  count| 37660708|  37660708|37660708|37660708|
|    min|         1|          1|       1|    1987|
|    max|         7|         31|      12|    2008|
+-------+----------+-----------+--------+--------+
```

h. graphs / charts to show the distribution of data

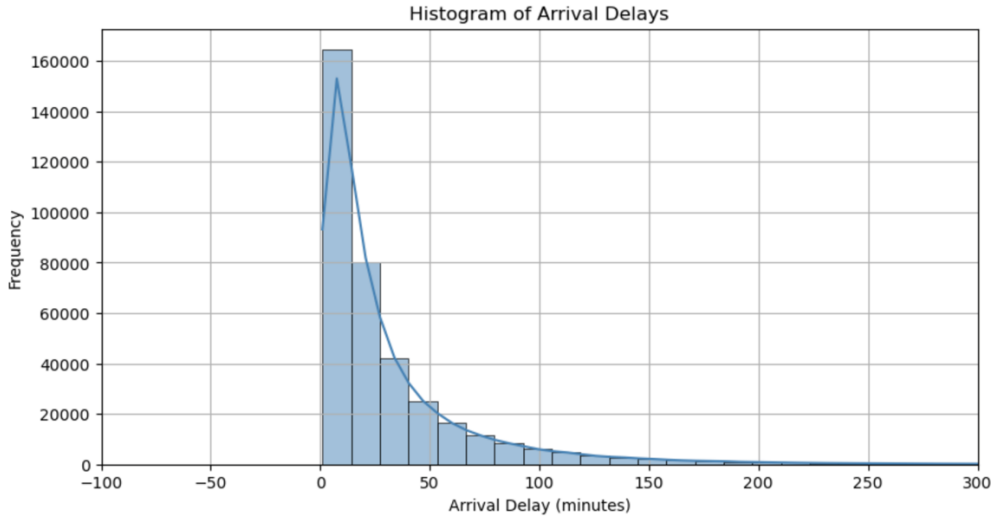
```
: delay_heatmap = sdf.groupBy("Month", "DayOfWeek") \
                     .avg("ArrDelay") \
                     .toPandas().pivot(index="Month", columns="DayOfWeek", values="avg(ArrDelay)")

plt.figure(figsize=(10, 6))
sns.heatmap(delay_heatmap, annot=True, fmt=".1f", cmap="coolwarm")
plt.title("Average Arrival Delay by Month & Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Month")
plt.show()
```



[20]:
```
# Sample and convert to pandas to keep it fast and memory-safe
pandas_df = sdf.select("ArrDelay").dropna().sample(False, 0.01).toPandas()

# Plot histogram
plt.figure(figsize=(10, 5))
sns.histplot(pandas_df["ArrDelay"], bins=100, kde=True, color="steelblue")
plt.title("Histogram of Arrival Delays")
plt.xlabel("Arrival Delay (minutes)")
plt.ylabel("Frequency")
plt.xlim(-100, 300)  # Optional: zoom into the most common delay range
plt.grid(True)
plt.show()
```

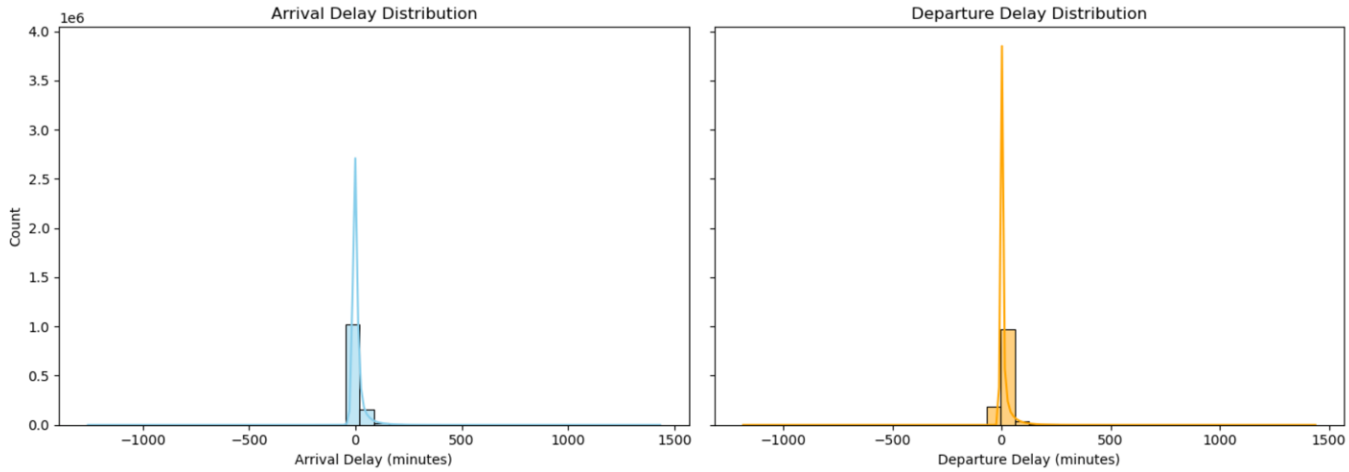



```
fig, axs = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

sns.histplot(pandas_df["ArrDelay"], kde=True, bins=40, color="skyblue", ax=axs[0])
axs[0].set_title("Arrival Delay Distribution")
axs[0].set_xlabel("Arrival Delay (minutes)")

sns.histplot(pandas_df["DepDelay"], kde=True, bins=40, color="orange", ax=axs[1])
axs[1].set_title("Departure Delay Distribution")
axs[1].set_xlabel("Departure Delay (minutes)")

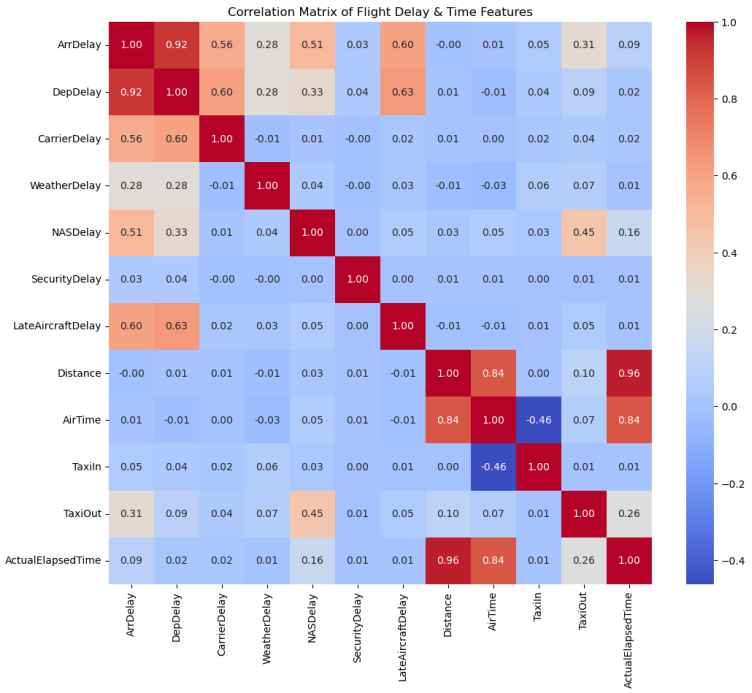
plt.tight_layout()
plt.show()
```



```
# Correlation Heatmap (for numeric relationships) Strong positive correlation (close to 1) → variables rise together
#Strong negative correlation (close to −1) → one rises, the other falls
#Near 0 → weak or no linear relationship
cols = [
    "ArrDelay", "DepDelay", "CarrierDelay", "WeatherDelay",
    "NASDelay", "SecurityDelay", "LateAircraftDelay",
    "Distance", "AirTime", "TaxiIn", "TaxiOut", "ActualElapsedTime"
]

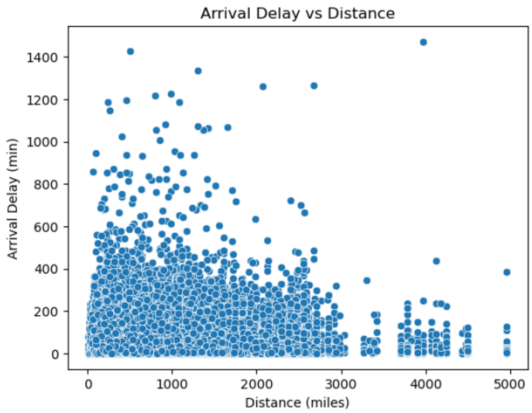
# Sample and convert to pandas
numeric_df = sdf.select(cols).dropna().sample(False, 0.01).toPandas()

plt.figure(figsize=(12, 10))
sns.heatmap(numeric_df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix of Flight Delay & Time Features")
plt.show()
```



[22]:
```
#Scatter plots (for checking relationships between two specific columns)
# Delay vs Distance
pair_df = sdf.select("ArrDelay", "Distance").dropna().sample(False, 0.01).toPandas()

sns.scatterplot(x="Distance", y="ArrDelay", data=pair_df)
plt.title("Arrival Delay vs Distance")
plt.xlabel("Distance (miles)")
plt.ylabel("Arrival Delay (min)")
plt.show()
```



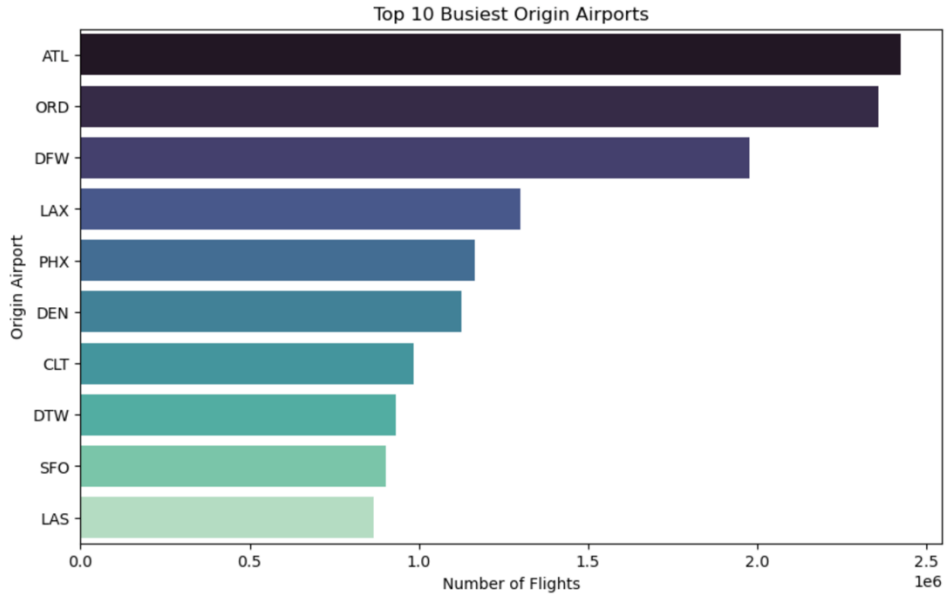
```
#Top 10 Busiest Airports (By Origin)
top_airports = sdf.groupBy("Origin").count().orderBy("count", ascending=False).limit(10).toPandas()

plt.figure(figsize=(10, 6))
sns.barplot(x="count", y="Origin", data=top_airports, palette="mako")
plt.title("Top 10 Busiest Origin Airports")
plt.xlabel("Number of Flights")
plt.ylabel("Origin Airport")
plt.show()
```

```
/tmp/ipykernel_11676/3497894759.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=Fals
e` for the same effect.

  sns.barplot(x="count", y="Origin", data=top_airports, palette="mako")
```



Appendix C Cleaned Data code

a. Find and Drop Missing Values

```
[12]: # Check to see if some of the columns have NULL values
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "AirTime", "ArrDelay", "DepDelay", "Cancelled",
```

```
[Stage 8:============================================================>(89 + 1) / 90]
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
|ActualElapsedTime|AirTime|ArrDelay|DepDelay|Cancelled|Distance|Month|Year|DayOfWeek|
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
|                0|      0|       0|       0|        0|       0|    0|   0|        0|
+-----------------+-------+--------+--------+---------+--------+-----+----+---------+
```

```
[13]: # Drop some of the records where the certain columns are empty (null or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

i. Dropping irrelevant columns (doesn't change number of records)

[13]:
```
# Drop some of the records where the certain columns are empty (null or na)
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

j. Parse columns to the correct datatype, may cause missing value (check and drop missing values) > print the schema again to check if the columns were dropped parsed

[15]:
```
# Parse columns to the appropriate data types
sdf = sdf.withColumn("ActualElapsedTime", sdf["ActualElapsedTime"].cast("integer"))
sdf = sdf.withColumn("ArrDelay", sdf["ArrDelay"].cast("integer"))
sdf = sdf.withColumn("DepDelay", sdf["DepDelay"].cast("integer"))
sdf = sdf.withColumn("Distance", sdf["Distance"].cast("float"))

# WeatherDelay, CarrierDelay, NASDelay, LateAircraftDelay, SecurityDelay – Parse them as float since they represent delay times
sdf = sdf.withColumn("CarrierDelay", sdf["CarrierDelay"].cast("integer"))
sdf = sdf.withColumn("NASDelay", sdf["NASDelay"].cast("integer"))
sdf = sdf.withColumn("LateAircraftDelay", sdf["LateAircraftDelay"].cast("integer"))
sdf = sdf.withColumn("SecurityDelay", sdf["SecurityDelay"].cast("integer"))
```

[16]:
```
# Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in ["ActualElapsedTime", "ArrDelay", "DepDelay", "Cancelled", "Distance"
```

```
[Stage 11:=========================================================>(89 + 1) / 90]
+-----------------+--------+--------+---------+--------+-----+----+---------+
|ActualElapsedTime|ArrDelay|DepDelay|Cancelled|Distance|Month|Year|DayOfWeek|
+-----------------+--------+--------+---------+--------+-----+----+---------+
|                0|       0|       0|        0|   71824|    0|   0|        0|
+-----------------+--------+--------+---------+--------+-----+----+---------+
```

[17]:
```
# Drop some of the records where the certain columns are empty (null or na) after parsing
sdf = sdf.na.drop(subset=["ArrDelay", "DepDelay", "Cancelled", "Distance","Month","Year","DayOfWeek"])
```

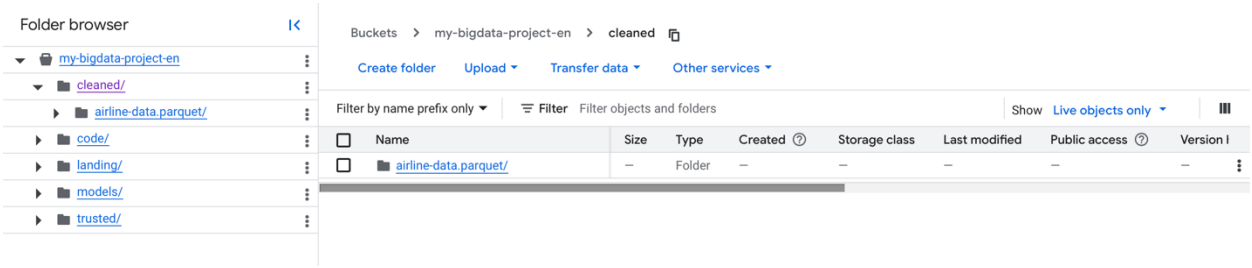
```
#print Schema for original and renamed columns
sdf.printSchema()
```

```
root
 |-- Actual_Elapsed_Time: integer (nullable = true)
 |-- Arrival_Delay: integer (nullable = true)
 |-- Cancelled: integer (nullable = true)
 |-- Carrier_Delay: integer (nullable = true)
 |-- Day_of_Week: integer (nullable = true)
 |-- Day_of_Month: integer (nullable = true)
 |-- Departure_Delay: integer (nullable = true)
 |-- Destination: string (nullable = true)
 |-- Distance: float (nullable = true)
 |-- Diverted: integer (nullable = true)
 |-- FlightNum: integer (nullable = true)
 |-- Late_Aircraft_Delay: integer (nullable = true)
 |-- Month: integer (nullable = true)
 |-- NASDelay: integer (nullable = true)
 |-- Origin: string (nullable = true)
 |-- Security_Delay: integer (nullable = true)
 |-- TailNum: string (nullable = true)
 |-- TaxiIn: string (nullable = true)
 |-- Airline: string (nullable = true)
 |-- Year: integer (nullable = true)
```

k. rename columns and removed spaces

```
# Saving the final dataset as a Parquet file
# Actually this will save as a folder with a file for each of the 90 partitions.
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"
sdf.write.parquet(cleaned_filename, mode='overwrite')
```

```
25/03/30 00:13:27 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
```



Milestone 4

The model predicts whether a flight will experience an arrival delay of more than 15 minutes based on features such as the airline, origin and destination airports, departure delay, scheduled arrival time, distance, and other flight-related information.

In this project, the dataset was first loaded from a cleaned Parquet file stored in a cloud bucket. Initial preprocessing included dropping irrelevant or leakage-prone columns, handling missing values, and engineering new features such as extracting the scheduled departure and arrival hours from the original time columns. A binary label was then created to classify flights as "Delayed" if the arrival delay exceeded 15 minutes.
The feature engineering process involved indexing categorical variables (Airline, Origin, Destination) using StringIndexer, one-hot encoding them with OneHotEncoder, assembling continuous features (like Departure_Delay, Distance), and scaling them with StandardScaler. These steps were built into a PySpark ML pipeline.
A logistic regression model was then trained using a CrossValidator to find the best hyperparameters through grid search, and the best model was selected based on the areaUnderROC evaluation metric. Performance metrics including accuracy, precision, recall, and F1 score were computed using a confusion matrix.

Results
accuracy = `0.8911829315627349`
precision = `0.9183108960745947`
recall =0.8840070646627571
f1_score = `0.9008325249720238`

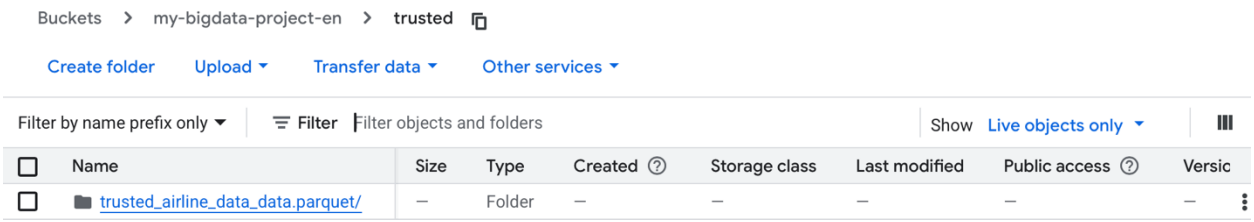
Challenges encountered during the process included handling poorly formatted time columns, addressing missing values in critical features, and ensuring that no target leakage occurred (by dropping features like Carrier_Delay). Building the full pipeline correctly to handle categorical and continuous variables separately and tuning hyperparameters for better classification performance were also key aspects of the work.
The final trained model was saved to cloud storage for future deployment or scoring.

Flight_features_treatment

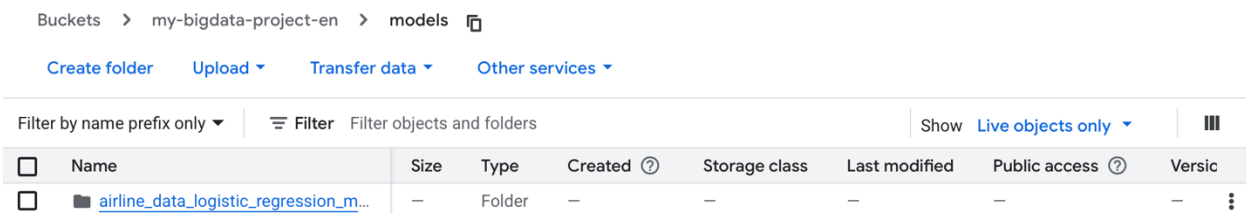
Model	Predict flight arrival delays using logistic regression to classify flights as "On Time" or "Delayed" based on whether the delay exceeds a certain percentage (15 minutes) of the total trip time						
	Column	Data Type	Variable Type	Keep or Drop	Indexer	Encoder	Scaler
Features	Actual_Elapsed_Time	integer	Continuous	Keep			StandardScaler
	Air_Time	string	Categorical	Drop			
	Arrival_Time	integer	Categorical	Drop			
	Scheduled_Arrival_Time	integer	Continuous	Keep (Dropped after converted to only hour)			StandardScaler
	Scheduled_Depture_Time	integer	Continuous	Keep (Dropped after converted to only hour)			
	Scheduled_Elapsed_Time	string	Categorical	Drop			
	Cancelled	integer	Categorical	Drop			
	Carrier_Delay	integer	Continuous	Drop			
	Day_of_Week	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Day_of_Month	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Departure_Delay	integer	Continuous	Keep			StandardScaler
	Depature_Time	sting	Categorical	Drop			
	Destination	string	Categorical	Keep	StringIndexer	OneHotEncoder	
	Distance	float	Continuous	Keep			StandardScaler
	Diverted	integer	Categorical	Drop			
	FlightNum	integer	Categorical	Drop			
	Month	integer	Categorical	Keep	StringIndexer	OneHotEncoder	
	Origin	sting	Categorical	Keep	StringIndexer	OneHotEncoder	
	TailNum	string	Categorical	Drop			
	TaxiIn	string	Continuous	Drop			
	TaxiOut	string	Continuous	Drop			
	Airline	string	Categorical	Keep	StringIndexer	OneHotEncoder	
	Year	integer	Categorical	Drop			
Label	Arrival_Delay	integer	Categorical	Keep	Manually derived		

Appendix D Source code

Trusted folder



Model Folder




```
#!/usr/bin/env python
# coding: utf-8
```

In[1]:

```
spark
```

In[2]:

```
import pandas as pd
import seaborn as sns
import matplotlib as plt
import matplotlib.pyplot as plt
import numpy as np
```

In[3]:

```
from pyspark.sql.functions import col, isnan, isnull, when, count, udf, size, split, year,
month, format_number, date_format, length
from pyspark.sql.types import IntegerType, DateType, StringType, StructType, DoubleType
from pyspark.ml.feature import VectorAssembler, StringIndexer, OneHotEncoder,
Normalizer, StandardScaler, HashingTF, IDF, Tokenizer, RegexTokenizer
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.regression import GeneralizedLinearRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
```

In[4]:

```
# Set up the path to a file
bucket = 'gs://my-bigdata-project-en'
landing_folder = f"{bucket}/landing/"
cleaned_folder = f"{bucket}/cleaned/"
trusted_folder = f"{bucket}/trusted/"
models_folder = f"{bucket}/models/"
cleaned_filename = f"{cleaned_folder}/airline-data.parquet"


# In[5]:


# Read in the Cleaned file
sdf = spark.read.parquet(cleaned_filename)


# In[6]:


# Check the schema
sdf.printSchema()


# In[7]:


# Drop some of the records not needed
sdf = sdf.drop("Arrival_Time","Air_Time",
"Scheduled_Elapsed_Time","Depature_Time","Carrier_Delay","Depature_Time","Diverted"
,"FlightNum","TailNum", "TaxiIn", "TaxiOut", "Year")


# In[8]:


# Check the schema
sdf.printSchema()


# In[9]:


# Check to see if some of the columns have NULL values after parsing
```

```
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in
["Actual_Elapsed_Time","Arrival_Delay","Scheduled_Arrival_Time","Scheduled_Depture_Ti
me","Cancelled","Day_of_Week","Day_of_Month","Departure_Delay","Destination","Dista
nce","Month","Origin","Airline"]] ).show()


# In[10]:


sdf.select("Actual_Elapsed_Time","Arrival_Delay","Scheduled_Arrival_Time","Scheduled_
Depture_Time","Cancelled","Day_of_Week","Day_of_Month","Departure_Delay","Destinati
on","Distance","Month","Origin","Airline").describe().show()


# In[11]:


from pyspark.sql.functions import lpad, substring

# Step 1: Pad the columns
time_columns = ["Scheduled_Depture_Time", "Scheduled_Arrival_Time"]
for col_name in time_columns:
   sdf = sdf.withColumn(col_name, lpad(col_name, 4, '0'))

# Step 2: Extract Hour columns
sdf = sdf.withColumn("Scheduled_Departure_Hour",
substring("Scheduled_Depture_Time", 1, 2).cast("int")) \
     .withColumn("Scheduled_Arrival_Hour", substring("Scheduled_Arrival_Time", 1,
2).cast("int"))


# In[12]:


# Check to see if some of the columns have NULL values after parsing
sdf.select([count(when(isnan(c) | col(c).isNull(), c)).alias(c) for c in
["Scheduled_Departure_Hour","Scheduled_Arrival_Hour"]] ).show()


# In[13]:


# Drop some of the records not needed
sdf = sdf.drop("Scheduled_Depture_Time", "Scheduled_Arrival_Time")
```

In[14]:

```
# Get the number of records in the dataframe
sdf.count()
```

In[15]:

```
sdf.select("Arrival_Delay").describe().show()
```

In[16]:

```
# IMPORTANT Last sanity check to remove any records that have NULL values in any of our important columns
list_of_columns = ["Actual_Elapsed_Time","Arrival_Delay","Day_of_Week","Day_of_Month","Departure_Delay","Destination","Distance","Month","Origin","Airline","Scheduled_Departure_Hour","Scheduled_Arrival_Hour"]
sdf = sdf.dropna(subset=list_of_columns, how="any")
```

In[17]:

```
# Create a label. =1 if the flight delayed is a 1, =0 if otherwise
from pyspark.sql.functions import when

sdf = sdf.withColumn("label", when(col("Arrival_Delay") > 14, 1).otherwise(0))
```

In[18]:

```
# Check the distribution of the label to make sure it is evenly balanced (approximately)
sdf.select("label").groupby("label").count().show()
```

In[19]:

```
# Create the Machine Learning pipeline
# 1. Categorical columns relevant to flight prediction
categorical_column_list = ["Airline", "Origin", "Destination"]
categorical_index_list = [c + "Index" for c in categorical_column_list]
categorical_encoded_list = [c + "Vec" for c in categorical_column_list]

# Indexing categorical columns
indexer = StringIndexer(inputCols=categorical_column_list,
outputCols=categorical_index_list, handleInvalid="keep")

# One-hot encoding the indexed categorical columns
encoder = OneHotEncoder(inputCols=categorical_index_list,
outputCols=categorical_encoded_list, dropLast=True)

# 2. Continuous/numeric columns available before flight arrival
continuous_column_list = [
  "Actual_Elapsed_Time",
  "Scheduled_Departure_Hour",
  "Scheduled_Arrival_Hour",
  "Day_of_Week",
  "Day_of_Month",
  "Departure_Delay",
  "Distance",
  "Month",
]

# 3. Assemble and scale continuous features
continuous_assembler = VectorAssembler(inputCols=continuous_column_list,
outputCol="continuousVector")
scaler = StandardScaler(inputCol="continuousVector", outputCol="continuousScaled")

# 4. Final assembler to put everything into one "features" vector
feature_columns = categorical_encoded_list + ["continuousScaled"]
assembler = VectorAssembler(inputCols=feature_columns, outputCol="features")


# In[20]:


# Test this much of the pipeline
# test_pipeline = Pipeline(stages=[indexer, encoder, continuous_assembler, scaler,
assembler])
```

```
# test_pipeline = Pipeline(stages=[indexer, encoder, continuous_assembler, scaler])
# transformed_sdf = test_pipeline.fit(sdf).transform(sdf)
# transformed_sdf.show(n=2, vertical=True)

# TODO
# Optionally, save this transformed_sdf to your Trusted folder as it will have all of the features.
# transformed_sdf.save.format(
trusted_filename = f"{trusted_folder}/trusted_airline_data_data.parquet"
sdf.write.parquet(trusted_filename, mode='overwrite')


# In[21]:


# Create a LogisticRegression Estimator
lr = LogisticRegression()


# In[22]:


# Assemble all of the steps into a pipline
machine_learning_pipeline = Pipeline(stages=[indexer, encoder, continuous_assembler, scaler, assembler, lr])


# In[23]:


# Split the sdf dataframe into training and test sets
trainingData, testData = sdf.randomSplit([0.7, 0.3])


# In[24]:


# Create a BinaryClassificationEvaluator to evaluate how well the model works
evaluator = BinaryClassificationEvaluator(metricName="areaUnderROC")


# In[25]:
```

```
# Create a grid to hold hyperparameters
grid = ParamGridBuilder()
grid = grid.addGrid(lr.regParam, [0.0, 0.5, 1.0])
grid = grid.addGrid(lr.elasticNetParam, [0, 1])

# Build the parameter grid
grid = grid.build()

# Show the number of models to be tested
print('Number of models to be tested: ', len(grid))


# In[26]:


# Create the CrossValidator
# Note the 'estimator' is pointing to the complete machine_learning_pipeline
cv = CrossValidator(estimator=machine_learning_pipeline,
          estimatorParamMaps=grid,
          evaluator=evaluator,
          numFolds=3
          )


# In[ ]:


# Use the CrossValidator to Fit the data
# This will train all of the models
all_models = cv.fit(trainingData)


# In[ ]:


# Choose the best model:
bestModel = all_models.bestModel


# In[ ]:


# Show the average metric on all of the model runs
print(f"Average metric: {all_models.avgMetrics}")
```

In[ ]:

```
# Use the bestModel to run the model on the testData
test_results = bestModel.transform(testData)
```

In[ ]:

```
# Evaluate the predictions
print(evaluator.evaluate(test_results))
```

In[ ]:

```
# Show the label and predicted label
test_results.select(["Airline", "Origin", "Destination",
                     'Arrival_Delay', 'Distance','label', 'prediction']).show(truncate=False)
```

In[ ]:

```
# Create the Confusion Matrix
cm = test_results.groupby('label').pivot('prediction').count().fillna(0).sort('label',
ascending=True).collect()
```

In[ ]:

```
# Function to calculate recall and precision
def calculate_recall_precision(cm):
    tn = cm[0][1]
    fp = cm[0][2]
    fn = cm[1][1]
    tp = cm[1][2]
    precision = tp / ( tp + fp )
    recall = tp / ( tp + fn )
    accuracy = ( tp + tn ) / ( tp + tn + fp + fn )
```

```
    f1_score = 2 * ( ( precision * recall ) / ( precision + recall ) )
    return accuracy, precision, recall, f1_score


# In[ ]:


accuracy, precision, recall, f1_score =  calculate_recall_precision(cm)


# In[ ]:


print(accuracy, precision, recall, f1_score)


# In[ ]:


cm


# In[ ]:


print("label,0.0,1.0\n", cm[0][0],",",cm[0][1],",",cm[0][2], "\n",
cm[1][0],",",cm[1][1],",",cm[1][2])


# In[ ]:


# Save the model to the models folder
model_path =  f"{models_folder}/airline_data_logistic_regression_model"
bestModel.write().overwrite().save(model_path)
```